C++ Programming Assignment

Q.1 Write a program to demonstrate different data types in C++ (int, float, double, char, bool) and print their sizes using sizeof() operator.

CODE :

```
#include <iostream>
using namespace std;

int main()
{
   int a = 10;
   float b = 3.5f;
   double c = 5.12345;
   char d = 'A';
   bool e = true;

   cout << "Size of int: " << sizeof(a) << endl;
   cout << "Size of float: " << sizeof(b) << endl;
   cout << "Size of double: " << sizeof(c) << endl;
   cout << "Size of char: " << sizeof(d) << endl;
   cout << "Size of bool: " << sizeof(e) << endl;
   return 0;
}
```

```
Size of int: 4
Size of float: 4
Size of double: 8
Size of char: 1
Size of bool: 1
```

Q.2 Write a program to perform arithmetic operations (+, -, *, /, %) on two numbers entered by the user.

CODE :

```
#include <iostream>
using namespace std;

int main()
{
   int a, b;
   cout << "Enter two numbers: ";
   cin >> a >> b;

   cout << "Addition: " << a + b << endl;
   cout << "Subtraction: " << a - b << endl;
   cout << "Multiplication: " << a * b << endl;
   cout << "Division: " << a / b << endl;
   cout << "Modulus: " << a % b << endl;

   return 0;
}
```

```
Enter two numbers: 3
4
Addition: 7
Subtraction: -1
Multiplication: 12
Division: 0
Modulus: 3
```

Q.3 Write a program to demonstrate relational and logical operators in C++

CODE :

```
#include <iostream>
using namespace std;

int main()
{
   int a = 0, b = 10;

   // Relational operators
   cout << "Relation operators ans is : \n";
   cout << (a == b) << endl;
   cout << (a != 10) << endl;
   cout << (a > b) << endl;
   cout << (a < b) << endl;
   cout << (a >= b) << endl;
   cout << (a <= b) << endl
       << endl;

   cout << "Logical operators ans is : \n";
   cout << ((a < b) && (b > 0)) << endl;
   cout << ((a > b) || (b > 0)) << endl;
   cout << (!a) << endl;

   return 0;
}
```

```
Relation operators ans is :
0
1
0
1
0
1

Logical operators ans is :
1
1
1
```

Q.4 Write a program to show the use of increment and decrement operators (pre and post).

CODE :

```
#include <iostream>
using namespace std;

int main()
{
   int a = 5;

   cout << "Pre Increment: " << ++a << endl;
   cout << "Post Increment: " << a++ << endl;
   cout << "After Post Increment: " << a << endl<<endl;

   cout << "Pre Decrement: " << --a << endl;
   cout << "Post Decrement: " << a-- << endl;
   cout << "After Post Decrement: " << a << endl;

   return 0;
}
```

```
Pre Increment: 6
Post Increment: 6
After Post Increment: 7

Pre Decrement: 6
Post Decrement: 6
After Post Decrement: 5
```

Q.5 Create a class Student with data members (roll no, name, marks). Use a constructor to initialize
and display the details.

CODE :

```
#include <iostream>
using namespace std;

class Student
{
   int roll;
   string name;
   float marks;

public:
   Student(int r, string n, float m)
   {
      roll = r;
      name = n;
      marks = m;
   }

   void display()
   {
      cout << roll << " " << name << " " << marks;
   }
};

int main()
{
   Student s(1, "Deepansh", 85);
   s.display();
   return 0;
}
```

```
1 Deepansh 85
```

Q.6 Create a class Rectangle that calculates area using a parameterized constructor.

CODE :

```
#include <iostream>
using namespace std;

class Rectangle
{
   int length, width;

public:
   Rectangle(int l, int w)
   {
      length = l;
      width = w;
      cout << "Area: " << length * width;
   }
};

int main()
{
   Rectangle r(5, 4);
   return 0;
}
```

```
Area: 20
```

Q.7 Write a program to demonstrate constructor overloading in a class Box.

CODE :

```
#include <iostream>
using namespace std;

class Box
{
   int length;

public:
   Box() { length = 0; }
   Box(int l) { length = l; }

   void display()
   {
      cout << "Length: " << length << endl;
   }
};

int main()
{
   Box b1;
   Box b2(10);

   b1.display();
   b2.display();
   return 0;
}
```

```
Length: 0
Length: 10
```

Q.8 Create a class Employee and demonstrate the working of a destructor by printing a message when the object is destroyed.

CODE :

```
#include <iostream>
using namespace std;

class Employee
{
public:
   Employee() { cout << "Constructor called\n"; }
   ~Employee() { cout << "Destructor called\n"; }
};

int main()
{
   Employee e;
   return 0;
}
```

```
Constructor called
Destructor called
```

Q.9 Write a program to demonstrate a copy constructor using a class Book.

CODE :

```
#include <iostream>
using namespace std;

class Book
{
   int price;

public:
   Book(int p)
   {
      price = p;
   }

   Book(const Book &b)
   {
      price = b.price;
   }

   void display()
   {
      cout << "Price: " << price;
   }
};

int main()
{
   Book b1(500);
   Book b2 = b1;

   b2.display();
   return 0;
}
```

```
Price: 500
```

Q.10 Create a class Complex and add two complex numbers using a friend function.

CODE :

```
#include <iostream>
using namespace std;

class Complex
{
   int real, imag;

public:
   Complex(int r, int i)
   {
      real = r;
      imag = i;
   }

   friend Complex add(Complex, Complex);

   void display()
   {
      cout << real << " + " << imag << "i";
   }
};

Complex add(Complex c1, Complex c2)
{
   return Complex(c1.real + c2.real, c1.imag + c2.imag);
}

int main()
{
   Complex c1(2, 3), c2(4, 5);
   Complex c3 = add(c1, c2);
   c3.display();
   return 0;
}
```

```
6 + 8i
```

Q.11 Create two classes and use a friend function to compare their private data members.

CODE :

```
#include <iostream>
using namespace std;

class A;
class B;

class A
{
   int x;
public:
   A(int a) { x = a; }
   friend void compare(A, B);
};

class B
{
   int y;
public:
   B(int b) { y = b; }
   friend void compare(A, B);
};

void compare(A a, B b)
{
   if (a.x > b.y)
      cout << "A is greater";
   else
      cout << "B is greater";
}

int main()
{
   A a(10);
   B b(5);
   compare(a, b);
   return 0;
}
```

```
A is greater
```

Q.12 Write a recursive function to calculate the factorial of a number.

CODE :

```
#include <iostream>
using namespace std;

int fact(int n)
{
   if (n == 0)
      return 1;
   return n * fact(n - 1);
}

int main()
{
   cout << fact(10);
   return 0;
}
```

```
3628800
```

Q.13 Write a recursive function to find the sum of first N natural numbers

CODE :

```
#include <iostream>
using namespace std;

int sum(int n)
{
   if (n == 0)
      return 0;
   return n + sum(n - 1);
}

int main()
{
   cout << sum(5);
   return 0;
}
```

```
15
```

Q.14 Write a program using an inline function to find the square of a number.

CODE :

```
#include <iostream>
using namespace std;

inline int square(int x)
{
   return x * x;
}

int main()
{
   cout << square(5);
   return 0;
}
```

```
25
```

Q.15 Create a class Circle and use an inline function inside the class to calculate the area of the Circle.

CODE :

```
#include <iostream>
using namespace std;

class Circle
{
   float radius;

public:
   Circle(float r)
   {
      radius = r;
   }

   inline float area()
   {
      return 3.14 * radius * radius;
   }
};

int main()
{
   Circle c(5);
   cout << "Area: " << c.area();
   return 0;
}
```

```
Area: 78.5
```